Zero-Shot 2D Object Detection Across Domains: COCO → KITTI

CMPE 249 Homework — 2D Object Detection, Track A (Experimental) Ronak Patel · November 2026
Code, configs, logs and raw metrics: https://github.com/Ron5474/cmpe-249-assignment-1

1. Objective and summary

This report evaluates six pretrained detector configurations from four detector families on two datasets, COCO val2017 and KITTI 2D object (driving, never seen in training), without any retraining. COCO is the training domain of the four closed-set models and the reference dataset for all six; the two open-vocabulary models were pretrained on broader grounding data (Section 2.1). It asks three questions: (1) which classes and object scales degrade most under domain shift, (2) how the families trade speed against accuracy, and (3) how open-vocabulary detectors compare with closed-set ones and how sensitive they are to prompt wording.

Before any cross-dataset number was trusted, the evaluation harness was checked by reproducing published COCO mAP for the four closed-set models (Section 4.1).

Main findings:

- **Every model loses a large share of its mAP** going from COCO to KITTI: roughly half for the four closed-set models and Grounding DINO (e.g. YOLO11x 55.8 → 24.0, RT-DETR 51.7 → 24.5 on the shared 3-class taxonomy), and 39% for YOLO-World from a lower COCO baseline.
- **YOLO11x's measured advantage shrinks on KITTI.** It beats YOLO11s by 8.1 mAP on COCO but only by 1.5 on KITTI. RT-DETR, not the largest YOLO, scores highest on KITTI.
- **Degradation is class-specific.** Vehicles keep ~80–90% of their AP, persons keep ~40%, and cyclists collapse to ~0.1 AP for every closed-set model. A label-definition mismatch between the datasets is a substantial confound for the cyclist result; these experiments do not establish that it explains the entire collapse.
- **Grounding DINO has much higher recall than the closed-set models** (KITTI AR100 55.0 vs 34–36) but lower mAP (21.9), and its accuracy depends strongly on prompt wording (21.9 vs 15.4 mAP).

2. Models and datasets

2.1 Model configurations

Configuration	Family	Checkpoint / source	Pretraining data	Framework
Faster R-CNN R50-FPN v2	Two-stage, anchor-based [1, 2]	`fasterrcnn_resnet50_fpn_v2` [10]	COCO	torchvision 0.23.0
YOLO11s, YOLO11x	One-stage, anchor-free, NMS [3, 11]	`yolo11s.pt`, `yolo11x.pt`	COCO	Ultralytics 8.3.203
RT-DETR R50	Query-based transformer, NMS-free [4, 5]	`PekingU/rtdetr_r50vd` [12]	COCO	HF Transformers 4.56.2
YOLO-World v2-s	Open-vocabulary, CNN [6]	`yolov8s-worldv2.pt`	Objects365, GoldG, CC3M-Lite [6]	Ultralytics 8.3.203
Grounding DINO-T	Open-vocabulary, transformer [7]	`IDEA-Research/grounding-dino-tiny` [12]	Objects365, GoldG, Cap4M [7]	HF Transformers 4.56.2

That is six configurations from four families. YOLO11s/x are two sizes of one family and are used for the within-family speed–accuracy comparison.

2.2 Datasets

Dataset	Split	Images	Notes
COCO 2017 [8]	val2017	5,000 (all)	Training domain of the four closed-set models; reference dataset for all six.
KITTI 2D object [9]	`training`	7,481 (all)	Karlsruhe dashcam, 1242×375, daytime. The test split has no public labels; the training split was never used by any model here, so it is a clean zero-shot test set.

2.3 Taxonomy mapping

COCO has 80 classes and KITTI has 8 plus `DontCare`. All cross-dataset numbers use the harness's `driving3` taxonomy, which folds labels and predictions *by class name* into three unified classes. **COCO is also scored in driving3**, so the COCO → KITTI drop reflects the domain change rather than a change in which classes are counted.

Unified class	COCO names	KITTI names
vehicle	car, truck, bus, motorcycle, train	Car, Van, Truck, Tram
person	person	Pedestrian, Person_sitting
cyclist	bicycle	Cyclist
(dropped)	the other 73 classes	Misc, DontCare

Two caveats are part of the analysis rather than hidden. COCO `bicycle` is the bike only, while KITTI `Cyclist` is one box around bike and rider (Section 6). KITTI `DontCare` regions are left as background, so detections inside them count as false positives.

3. Experimental setup

3.1 Harness and protocol

Evaluation uses the course's `ngdet` harness [13], pinned as a git submodule at commit `8418fc1`. A wrapper (`scripts/eval.py`) adds what the harness lacks: a `coco80` taxonomy for the reproduction check, per-image saving of predicted and ground-truth boxes for failure analysis, and peak GPU memory and timing per run. Detectors are built through `scripts/detectors.py`, which corrects two adapter behaviours described in Section 3.4. ngdet itself is not modified.

Setting	Value
Metric	pycocotools COCOeval, bbox, maxDets = 100
Score threshold before scoring	**0.001** for all models, including Faster R-CNN's internal `roi_heads.score_thresh` (the tutorial default 0.3 truncates the PR curve and understates mAP). Grounding DINO is scored from its prompt tokens (Section 3.4), so no `text_threshold` applies and the same 0.001 floor holds.
Images	all images in both datasets (`--max-images 0`)
Input resolution	YOLO11 / YOLO-World: 640 letterbox (Ultralytics default). Faster R-CNN: shorter side 800, max 1333 (torchvision default). RT-DETR: 640×640 (HF processor). Grounding DINO: shorter side 800, max 1333 (HF processor). Boxes are mapped back to original image coordinates by each adapter.
NMS	YOLO: Ultralytics default IoU 0.7, max_det 300. Faster R-CNN: torchvision default IoU 0.5, 100 per image. RT-DETR (300 queries) and Grounding DINO (900 queries): no NMS.
Grounding DINO scoring	Token-to-class scores following the official COCO evaluator: class probability = mean of the class's token probabilities, top 300 query–class pairs per image (one query may predict several classes). For multi-term classes we average tokens within a term and take the maximum term within a class (our adaptation; ADR-0003).
Open-vocab prompts	Grounding DINO: curated synonyms `car. truck. bus. van. person. pedestrian. bicycle. cyclist .` (main run) and bare class names `vehicle. person. cyclist .` (ablation). YOLO-World: the bare class names (`set_classes`).
Latency	batch 1, one fixed KITTI frame (1242×375), 10 warm-up + 50 timed calls of the full `predict()` (pre-processing, forward, post-processing, label folding), `torch.cuda.synchronize()` around each call; peak memory from `torch.cuda.max_memory_allocated`.

3.2 Hardware and software

Item	Value
GPU	NVIDIA GeForce RTX 5090, 32 GB (rented on Vast.ai), driver 595.99
Software	Python 3.12.15, PyTorch 2.8.0+cu128, torchvision 0.23.0, Transformers 4.56.2, Ultralytics 8.3.203, pycocotools 2.0.8; all pinned in `uv.lock`
OS	Linux 6.8 (container), glibc 2.39
Third host (RT-DETR COCO-80 rerun only)	NVIDIA RTX A2000 Laptop GPU, 4 GB; same pinned software via `uv.lock`; Linux 6.6 (WSL2)

The results come from two rented RTX 5090 instances with the same software. The original run (instance 1) produced every accuracy result for YOLO11s, YOLO11x, RT-DETR and YOLO-World. The corrected reruns (instance 2) produced every Faster R-CNN accuracy result (code `0f9be9f`), every Grounding DINO accuracy result (code `51dad16`) and **all latency and memory numbers** (code `51dad16`), so the latency comparison shares one machine (AMD EPYC 7V12 host CPU). RT-DETR's COCO-80 reproduction was rerun a third time, after the class-name fix, on a laptop RTX A2000 (code `51dad16-dirty`, the working tree with that fix); fp32 accuracy does not depend on the GPU, and no latency was measured there. Results written by the corrected reruns record their GPU and code version inside `metrics.json`; the eleven retained instance-1 results predate those fields and are covered by that run's `env.json` and `commands.log`. Every run writes an environment file (`env.json`, or timestamped `env_*.json` for the reruns) with versions and the exact `argv`.

3.3 Exact commands

Every command is recorded in `results/logs/commands.log`. The runs were:

```
# instance 1: scripts/run_all.sh (all six models)
eval.py --exp repro_coco80  --taxonomy coco80        --models <4 closed-set models> --datasets coco
eval.py --exp driving3      --taxonomy driving3      --models <all six>             --datasets coco
kitti
eval.py --exp driving3_bare --taxonomy driving3_bare --models gdino:IDEA-Research/grounding-dino-tiny
--datasets kitti
latency.py --models <all six> --accel fp32 fp16
# instance 2: scripts/rerun.sh (Faster R-CNN, code 0f9be9f), then scripts/rerun_gdino.sh (code
51dad16)
eval.py  same three experiments, Faster R-CNN and Grounding DINO only
compare_gdino_postprocess.py --dataset {coco,kitti} --max-images 500
latency.py --models <all six> --accel fp32 fp16
# laptop (RTX A2000): scripts/rerun_rtdetr.sh (COCO-80 aliases)
eval.py --exp repro_coco80 --taxonomy coco80 --models hf_detr:PekingU/rtdetr_r50vd --datasets coco
```

All eval.py runs use `--max-images 0` (every image) and `uv run scripts/…`.

To reproduce from scratch: `git clone --recurse-submodules …`, `uv sync`, `bash scripts/download_data.sh`, then `bash scripts/run_all.sh` (all images by default). `uv run scripts/analyze.py` regenerates the figures and writes the tables to `report/tables.md`, from which the tables below are copied. The failure figure also needs the five KITTI frames and their label files copied into `data/kitti_figs/` (chosen by `scripts/pick_failures.py`).

3.4 Corrections after review

A review of the first version found two adapter defects, both corrected in `scripts/detectors.py` (tested by `scripts/test_fixes.py`) and rerun. A second review of the corrected version found a third, in the wrapper's `coco80` taxonomy (`scripts/eval.py`):

- **Faster R-CNN** silently kept torchvision's internal 0.05 score threshold, so it never ran at the stated 0.001 floor. The wrapper now sets `roi_heads.score_thresh` to 0.001.
- **Grounding DINO**'s phrase-to-class fallback in ngdet tested `phrase in class_name`. The empty string is a substring of every string, so every box whose phrase was empty (every box scoring below `text_threshold`) was labelled `vehicle`, and multi-term phrases such as `car truck` were dropped.
- **RT-DETR on COCO-80**: the `PekingU/rtdetr_r50vd` checkpoint names six classes VOC-style in `id2label` (`motorbike`, `aeroplane`, `sofa`, `pottedplant`, `diningtable`, `tvmonitor`). The `coco80` taxonomy

was built from the COCO annotation names with no aliases, so those six classes' detections were discarded before scoring and each had AP 0 in the first reproduction run. The taxonomy now carries the six aliases (`COCO80_ALIASES` in `scripts/eval.py`, checked by `test_fixes.py`), and only RT-DETR's COCO-80 reproduction was rerun; the `driving3` taxonomy already maps `motorbike` to vehicle and ignores the other five, so the cross-dataset results were unaffected.

The first Grounding DINO fix kept phrase decoding but dropped empty and mixed-class phrases. That removed the mislabelling but introduced two new problems: phrases only exist above `text_threshold` = 0.25, which acted as a score floor, and mixed phrases such as `person cyclist`, which were common on COCO images with people, were dropped even when the box covered a real person. (An informal count on a few dozen COCO images put mixed phrases at roughly a fifth of non-empty phrases; that diagnostic was not saved, so no exact share is claimed. The three-way comparison below measures the aggregate effect of each rule, not whether individual dropped boxes were true positives.) The final version therefore drops phrase decoding and scores classes from the prompt tokens as described in Section 3.1 (ADR-0003). Scoring the same forward passes with all three post-processors shows the difference (first 500 images of each dataset, diagnostic only):

Post-processor (driving3)	COCO mAP	COCO AR100	KITTI mAP	KITTI AR100
ngdet phrase mapping (empty → vehicle)	36.7	61.3	17.9	42.8
phrase parser, empty and mixed dropped (first fix)	31.7	48.2	19.7	37.6
token-to-class scoring (used in this report)	**40.0**	**72.4**	**21.7**	**55.9**

The three YOLO-family models were unaffected and were not rerun. The original results are in `results/v1_original/` and the phrase-parser results in `results/v2_phrase_parser/`. Predictions are now saved in full; the first run kept only the top 100 per image, which is not the same as COCOeval's 100 per class, so those files cannot always reproduce the original metrics exactly.

4. Results

4.1 Reproduction check (COCO val2017, all 80 classes)

Model	Ours mAP	Published	Δ	AP50	AP75	AP_S	AP_M	AP_L	AR100
Faster R-CNN v2	46.2	46.7 [10]	−0.5	66.7	50.5	24.4	45.1	59.0	62.6
YOLO11s	45.0	47.0 [11]	−2.0	61.3	48.6	21.2	42.8	60.2	61.4
YOLO11x	52.9	54.7 [11]	−1.8	69.2	57.6	28.4	52.4	66.9	68.2
RT-DETR R50	52.3	53.1 [4]	−0.8	70.2	56.9	27.2	50.5	67.7	71.8

All four land below their published values. Two harness behaviours are plausible contributors. First, ngdet re-creates the ground truth with `iscrowd = 0` for every box, so COCO crowd regions become ordinary objects: detections on them are matched or counted as false positives like any other box, and a missed crowd region counts as a miss, whereas COCOeval normally ignores crowd regions. This can affect models differently, so it is not a uniform offset. Second, ngdet computes area from the box rather than COCO's mask area, which mainly changes the small/medium/large results. The YOLO gap is also consistent with `predict()` using plain letterbox inference rather than the rectangular-batch validation Ultralytics uses for its published numbers. I did not separate these effects.

RT-DETR's first reproduction scored 48.2 (−4.9). That gap was not a harness bias but the class-name defect in Section 3.4: six classes had AP 0 because their detections were discarded. With the aliases it scores 52.3 (−0.8), the closest of the four to its published value, so its COCO number is now compared with the others. The first result is kept in `results/v1_original/repro_coco80/`. Using identical code on both datasets does not make annotation-related biases cancel, so the COCO → KITTI comparisons below are measurements under one protocol rather than bias-free estimates.

4.2 Cross-dataset results (driving3 taxonomy)

Model	Data	mAP	AP50	AP75	AP_S	AP_M	AP_L	AR100
Faster R-CNN v2	COCO	49.3	75.5	52.4	28.8	50.0	66.2	62.0
	KITTI	23.8	44.2	23.5	12.1	23.2	32.0	36.4
YOLO11s	COCO	47.6	69.3	51.0	20.1	48.8	70.4	62.0
	KITTI	22.5	42.9	21.5	10.5	21.3	31.5	34.1
YOLO11x	COCO	**55.8**	**77.3**	**60.6**	**29.5**	**58.3**	**75.5**	68.1
	KITTI	24.0	44.6	23.5	**13.5**	22.8	31.4	34.1
RT-DETR R50	COCO	51.7	75.4	54.2	26.6	52.7	72.6	67.1
	KITTI	**24.5**	**45.0**	**23.8**	13.2	**23.4**	**32.7**	36.2
YOLO-World v2-s	COCO	34.4	50.4	36.3	13.8	33.5	55.8	52.7
	KITTI	21.0	39.9	20.2	9.9	20.5	31.1	34.6
Grounding DINO-T	COCO	42.3	58.7	46.8	23.5	46.2	58.6	**70.5**
	KITTI	21.9	41.5	21.1	13.0	23.2	32.5	**55.0**

Per-class AP@[.5:.95], COCO → KITTI

Model	vehicle	person	cyclist
Faster R-CNN v2	53.4 → 45.7	56.6 → 25.6	38.0 → 0.1
YOLO11s	51.6 → 44.5	56.2 → 22.9	35.0 → 0.1
YOLO11x	59.5 → 47.6	62.7 → 24.3	45.1 → 0.1
RT-DETR R50	56.3 → 49.2	59.3 → 24.1	39.5 → 0.1
YOLO-World v2-s	37.5 → 40.6	51.7 → 21.6	13.9 → 0.8
Grounding DINO-T	47.3 → 43.9	53.6 → 18.9	25.8 → 2.9

4.3 Latency and memory (instance 2: RTX 5090, AMD EPYC 7V12 host, batch 1, KITTI frame)

Model	fp32 ms	fp32 p90	fp32 FPS	fp32 peak MB	fp16 ms	fp16 peak MB
YOLO11s	12.75	12.86	78.5	87	13.32	48
YOLO-World v2-s	14.22	14.67	70.3	933	14.55	888
YOLO11x	19.41	19.57	51.5	366	20.10	233
Faster R-CNN v2	21.04	37.53	47.5	473	20.22	245
RT-DETR R50	44.51	44.84	22.5	297	45.81	341
Grounding DINO-T	125.53	130.44	8.0	1,621	n/a*	

*The ngdet Grounding DINO adapter accepts an `accel` argument but never applies it, so its "fp16" run was actually fp32 (122.3 ms, 1,621 MB) and is not reported as fp16.

Host comparison. The first run measured latency on instance 1 (same GPU model, different host). For the four models whose code did not change between runs, the same measurement was 2.2–2.7× slower on instance 2:

Model (fp32 median ms)	Instance 1	Instance 2	Ratio
YOLO11s	5.84	12.75	2.18×
YOLO11x	8.05	19.41	2.41×
RT-DETR R50	16.57	44.51	2.69×
YOLO-World v2-s	6.44	14.22	2.21×

5. Technical analyses

5.1 Cross-dataset generalization: which classes and scales degrade most?

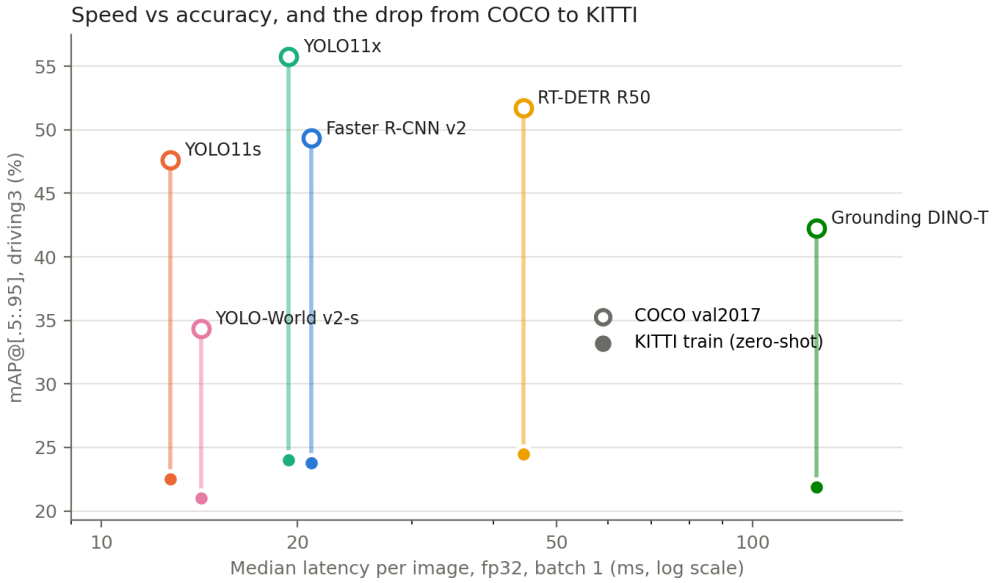


Figure 1. Median fp32 latency (batch 1, log scale) against driving3 mAP. Hollow markers: COCO val2017; filled: KITTI. The vertical line is each model's COCO → KITTI change.

Overall. The clearest within-family observation: YOLO11x's 8.1-point lead over YOLO11s on COCO shrinks to 1.5 points on KITTI. The measured advantage of the larger model largely does not carry over to this domain; these experiments do not show why.

By class. Vehicles transfer best (they keep 80–87% of COCO AP for the closed-set models). KITTI is dominated by cars seen from a dashcam, which is also a common COCO viewpoint for cars. **Persons lose more than half their AP** (e.g. 56.6 → 25.6 for Faster R-CNN). KITTI pedestrians are small, often in groups, partially occluded by parked cars, and seen side-on at a distance, which COCO's photographer-framed people rarely are (Figure 3, frame 004139). **Cyclist AP falls to 0.1 for the closed-set models**; Section 6 shows a definition mismatch that is a substantial confound for this result.

By scale. For the closed-set models, KITTI AP falls to roughly 40–50% of COCO AP in each size bin, not just for small objects (Figure 2). In absolute terms small objects are worst (AP_S ≈ 10–13 on KITTI), but they were already weak on COCO, so their relative loss is similar. Input resizing differs by model: YOLO and RT-DETR shrink a 1242×375 KITTI frame to 640 pixels wide, while Faster R-CNN and Grounding DINO enlarge it to about 1333×402. A resolution effect therefore cannot be uniform across models, and the loss on large objects suggests appearance and context differences (viewpoint, occlusion, lighting) also contribute; I did not test this.

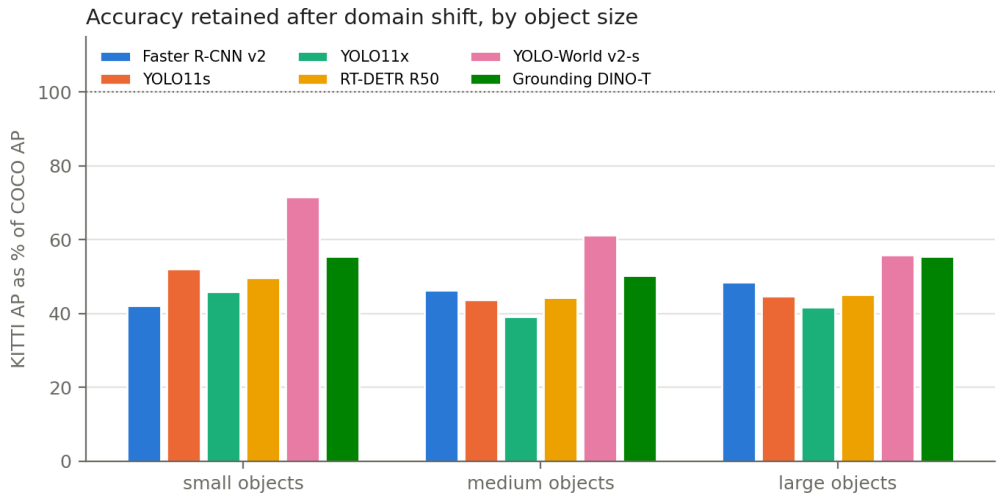


Figure 2. KITTI AP as a percentage of COCO AP, per COCO size bin (small < 32², medium < 96², large ≥ 96² pixels of box area).

Absolute accuracy and relative retention differ. YOLO-World has the smallest proportional decline (it keeps about 72%, 61% and 56% of its small, medium and large AP), but its COCO baseline is low, and its absolute KITTI mAP is below every closed-set model.

5.2 Architecture comparison and speed–accuracy trade-off

Two-stage vs one-stage vs query-based. On COCO, YOLO11x scores highest (52.9 on all 80 classes, RT-DETR second at 52.3; 55.8 vs 51.7 in driving3). On KITTI, **RT-DETR scores highest (24.5 mAP)**, and Faster R-CNN, an older two-stage design, is level with YOLO11x (23.8 vs 24.0). The closed-set KITTI scores span only 2 points (22.5–24.5), so these rankings are close. One plausible reason is post-processing. RT-DETR's set prediction needs no NMS, while YOLO and Faster R-CNN suppress overlapping boxes. KITTI scenes have many overlapping parked cars and grouped pedestrians, where NMS can remove true positives. I did not isolate this effect experimentally, so it is a hypothesis.

Speed. On instance 2 at batch 1, the YOLO-based models are fastest (12.8–19.4 ms), Faster R-CNN is close (21.0 ms), RT-DETR is slower (44.5 ms), and Grounding DINO is roughly an order of magnitude slower than the YOLOs (126 ms, 8 FPS). On KITTI the speed–accuracy frontier is flat: YOLO11s is 2.0 mAP behind the highest-scoring model (RT-DETR) while running about 3.5× faster (12.8 vs 44.5 ms). **Under this domain shift, the small model was nearly as accurate as the larger ones at a fraction of the latency.**

fp16 reduced measured latency only for Faster R-CNN, and only slightly (21.0 → 20.2 ms median; its p90 of about 37 ms shows this measurement is noisy). The YOLOs, YOLO-World and RT-DETR were marginally slower in fp16. Peak memory fell with fp16 for Faster R-CNN (473 → 245 MB) and the YOLO models, but rose for RT-DETR (297 → 341 MB).

Host sensitivity. The same GPU model gave 2.2–2.7× different latencies on two hosts (Section 4.3). The timing covers the whole `predict()` call, including CPU-side image conversion and resizing, so a plausible explanation is substantial CPU overhead at batch 1, which would also explain why fp16 barely changed latency on instance 2 (on instance 1, the original Faster R-CNN measurement did improve with fp16, 12.0 → 9.3 ms). GPU clocks, contention and transfer speed could also contribute; I did not profile the individual stages, so this is a hypothesis. The practical point is that full-pipeline latency numbers are only comparable on the same host.

Memory is small for all closed-set models (≤ 0.5 GB). The open-vocabulary models are the heaviest: YOLO-World uses 0.9 GB despite its small detector, and Grounding DINO uses 1.6 GB.

5.3 Closed-set vs open-vocabulary detection

On KITTI mAP, the open-vocabulary models were not more robust. Grounding DINO-T (21.9) and YOLO-World v2-s (21.0) both score below every closed-set model (22.5–24.5). Their relative declines differ: Grounding DINO loses 48% of its COCO mAP, similar to the closed-set models, while YOLO-World loses 39% from a much lower COCO baseline.

Grounding DINO has much higher recall. Its AR100 is 55.0 on KITTI and 70.5 on COCO, against 34–36 and 62–68 for the closed-set models. Its AP50 (41.5) and AP75 (21.1) are slightly below the closed-set range. High recall combined with lower AP means its additional correct detections are not ranked high enough to raise precision, but these metrics do not identify whether score calibration, localization or false positives are responsible. The comparison is also affected by different output caps: Grounding DINO keeps 300 query–class pairs per image, Faster R-CNN 100 boxes, and YOLO up to 300 boxes, so AR is not measured under identical candidate budgets.

Open-vocabulary detectors score higher on the label-mismatch class. On cyclist, Grounding DINO reaches 2.9 AP and YOLO-World 0.8, against 0.1 for the closed-set models. A text prompt such as `cyclist` can describe the bike-plus-rider concept, while a closed-set head can only emit COCO's `bicycle`. The absolute values are still very low.

Prompt wording matters a great deal.

Grounding DINO-T prompt (KITTI)	mAP	AR100	vehicle	person	cyclist
curated: `car. truck. bus. van. person. pedestrian. bicycle. cyclist .`	**21.9**	**55.0**	**43.9**	18.9	2.9
bare: `vehicle. person. cyclist .`	15.4	51.7	15.8	**25.8**	**4.6**

Prompting with concrete nouns (`car. truck. bus. van`) instead of the abstract word `vehicle` nearly triples vehicle AP (15.8 → 43.9). The effect reverses for the other classes: bare `person` scores higher than `person. pedestrian` (25.8 vs 18.9), and bare `cyclist` higher than `bicycle. cyclist` (4.6 vs 2.9). I did not test why; one possibility for cyclist is that `bicycle` raises scores for boxes around parked bikes, which KITTI does not label. **No single prompt style was best for every class**, so open-vocabulary results should always be reported with their exact prompt and scoring rule.

6. Failure analysis



Figure 3. Failure cases on KITTI. Green: ground truth in driving3. Blue: YOLO11x detections with score ≥ 0.3. Orange: Grounding DINO-T (frame 000273 only). White dashed: KITTI DontCare regions. Frames were chosen by a heuristic (`scripts/pick_failures.py`: IoU ≥ 0.5 overlap, ignoring predicted class and one-to-one matching) as strong examples of each failure type, not as a representative sample; numbers match the list below.

1. **Small, distant objects (003554).** A row of cars at the far end of the road is under 25 pixels tall after resizing. Most are found, but several are missed or poorly localized, which matches the low AP_S.

2. **Cyclist definition mismatch (002920).** YOLO11x labels many **parked bicycles** as `cyclist`, because COCO `bicycle` is any bike. KITTI labels only ridden bikes, and boxes rider and bike together. Each parked bike becomes a false positive, and the real cyclist's rider is predicted as a separate `person`. This mismatch is a substantial confound for the ~0.1 cyclist AP of the closed-set models; testing how much of the collapse it explains would need a rule that merges `person` and `bicycle` boxes, or fine-tuning.
3. **Crowded, occluded pedestrians (004139).** A group of pedestrians on the left is partially occluded and close together. Several are missed, consistent with NMS suppressing adjacent boxes and with the large person-AP drop.
4. **Penalties from the evaluation itself (002526).** Many "false positives" are parked bicycles (the cyclist mismatch again) or detections inside DontCare regions. Because ngdet treats DontCare as background (ADR-0001), a detector that correctly sees a distant, unlabeled object is penalized. This lowers KITTI precision for all models, and somewhat more for models that detect more distant objects.
5. **Open-vocabulary recall (000273).** Grounding DINO recovers several distant cars that YOLO11x misses, but several of its boxes fall in DontCare regions or around the far car row rather than individual cars, consistent with its high-AR, lower-AP profile.

Limitations of this study.

- KITTI numbers are COCO-style AP over size bins, not the official KITTI Easy/Moderate/Hard metric (IoU 0.7 for cars), so they are not comparable with the KITTI leaderboard.
- The harness converts COCO crowd regions into normal ground truth and uses box area. Both can affect COCO results, and not necessarily equally across models.
- RT-DETR's COCO-80 reproduction was rerun on a third machine (a laptop RTX A2000, Section 3.2) from the working tree before the alias fix was committed, so its `code_version` reads `51dad16-dirty`; the fix is the only change in that tree.
- Latency is the full adapter pipeline at batch 1, not pure network time, and it varied 2.2–2.7× between two hosts with the same GPU model; Grounding DINO fp16 was not actually applied by the adapter.
- Grounding DINO's synonym pooling (mean within a term, max within a class) is our adaptation of the official evaluator, and its 300-pair output cap differs from the other models' caps.
- The accuracy results come from two instances (Section 3.2); accuracy should not depend on the host, but this was not re-verified by rerunning an unchanged model.
- A third, condition-diverse driving dataset (nuImages: night, rain, Singapore) was planned but dropped for time, so all driving results come from one city in daytime.
- No fine-tuning was done, so the analysis cannot say how much of the gap a few hundred KITTI labels would recover.

7. Conclusions

1. **COCO accuracy was a poor predictor of KITTI accuracy here.** The closed-set models lose roughly half their mAP on KITTI and compress into a 2-point band. The highest-scoring KITTI model (RT-DETR) is not the highest-scoring COCO model (YOLO11x).
2. **The losses are class-specific.** Vehicles transfer well, persons lose more than half their AP, and cyclists collapse. COCO and KITTI define the cyclist class differently, a substantial confound, so part of the measured gap may be a definition gap rather than a perception gap.
3. **Under this domain shift, the small model was nearly as good.** YOLO11s is 2.0 mAP behind the highest-scoring KITTI model at about 3.5× lower latency. fp16 barely changed batch-1 latency on instance 2 (the original Faster R-CNN measurement on instance 1 did improve, 12.0 → 9.3 ms), and full-pipeline latency depended strongly on the host.
4. **Grounding DINO showed much higher recall but lower mAP than the closed-set models**, ran about 10× slower than YOLO11s, and its per-class accuracy changed by nearly 3× with prompt wording. How it is post-processed also matters: on the same forward passes, three post-processors gave KITTI mAP between 17.9 and 21.7.

5. **Validating the pipeline was worth it.** After three corrections the reproduction check puts all four closed-set models within 2 points of their published values. Two external reviews found the defects: two adapter faults that changed the Faster R-CNN and Grounding DINO results, and a class-name mismatch that had silently discarded six of RT-DETR's classes and produced a 4.9-point gap that the first version called unexplained. All three were corrected and rerun (Section 3.4).

AI tool usage

AI tools were used as the assignment encourages. Claude (Anthropic) helped plan the experiment, write and debug the wrapper, analysis and report scripts, inspect the harness source, analyse the results, and draft this report. A second model (GPT) reviewed the work, found the two adapter defects in Section 3.4, and corrected my initial description of the official Grounding DINO evaluator; a second review of the corrected version found the RT-DETR class-name defect and several wording errors. I verified each point against the code and saved predictions before fixing. I ran all GPU experiments on rented instances, checked the numbers in this report against the saved `metrics.json` files, and am responsible for the content.

References

Papers

- [1] S. Ren, K. He, R. Girshick, J. Sun. "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks." NeurIPS 2015.
- [2] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, S. Belongie. "Feature Pyramid Networks for Object Detection." CVPR 2017.
- [3] J. Redmon, S. Divvala, R. Girshick, A. Farhadi. "You Only Look Once: Unified, Real-Time Object Detection." CVPR 2016.
- [4] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, J. Chen. "DETRs Beat YOLOs on Real-time Object Detection." CVPR 2024.
- [5] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, S. Zagoruyko. "End-to-End Object Detection with Transformers." ECCV 2020.
- [6] T. Cheng, L. Song, Y. Ge, W. Liu, X. Wang, Y. Shan. "YOLO-World: Real-Time Open-Vocabulary Object Detection." CVPR 2024.
- [7] S. Liu et al. "Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection." ECCV 2024.
- [8] T.-Y. Lin et al. "Microsoft COCO: Common Objects in Context." ECCV 2014.
- [9] A. Geiger, P. Lenz, R. Urtasun. "Are we ready for Autonomous Driving? The KITTI Vision Benchmark Suite." CVPR 2012.

Official repositories and documentation

- [10] PyTorch torchvision, "Models and pre-trained weights: `fasterrcnn_resnet50_fpn_v2`" (COCO box mAP 46.7). https://pytorch.org/vision/stable/models.html
- [11] Ultralytics, "YOLO11" documentation (COCO val mAP 47.0 for YOLO11s, 54.7 for YOLO11x). https://docs.ultralytics.com/models/yolo11/
- [12] Hugging Face Transformers documentation, RT-DETR and Grounding DINO model pages. https://huggingface.co/docs/transformers
- [13] K. Liu, DeepDataMiningLearning, `ngdet` tutorial and harness (commit `8418fc1`). https://github.com/lkk688/DeepDataMiningLearning/blob/main/DeepDataMiningLearning/ngdet/TUTORIAL.md